Renewable Energy Orchestrator

An agentic AI that dispatches solar, wind and batteries every 15 minutes, reacts to alerts, and explains itself

5 solar farms

3 wind farms

2 batteries

ET x Accenture AI Hackathon, Agentic Edition. Problem 4: Utilities

Created By Charu Jain

The problem: orchestrate renewables, batteries and the market every 15 minutes

Renewable energy generation, electricity prices, and demand change every 15 minutes. A utility managing five solar farms, three wind farms, two batteries, industrial loads, and grid connections must continuously decide when to charge, discharge, buy, sell, or curtail electricity.
The challenge is to balance cost, carbon emissions, clean energy utilization, and supply reliability while adapting to changing conditions and operational alerts. The solution must respond to changing conditions, make informed decisions, and explain the trade-offs behind its actions.

Key Challenges:

- **Dynamic Conditions:** Changes in renewable generation, demand, prices, and equipment availability.
- **Operational Decisions:** Charging, discharging, buying, selling, reserving, and curtailing power.
- **Competing Objectives:** Balancing cost, carbon emissions, energy waste, battery degradation, and reliability.
- **Alerts and Explainability:** Adapting to operational alerts and clearly explaining why each action was taken.

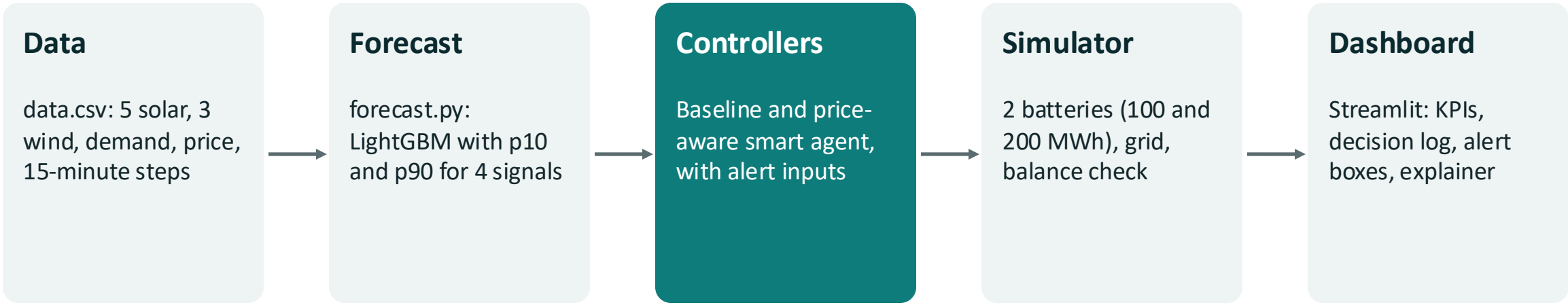
Our solution: a transparent renewable dispatch agent

Every 15 minutes, the agent uses renewable generation, demand, forecasts, battery state, and alerts to choose a dispatch action, then records and explains the result.



Design rule: numbers are decided by a policy we can test and verify. The LLM only turns operator text into structured inputs, and decisions into words.

Architecture: a numeric policy decides, the LLM explains



LLM layer alert parser (outage, storm), decision explainer, keyword and template fallback. Interprets and explains, never decides.

Safety check every step: generation + discharge + grid buy = demand + charge + grid sell + curtailment. No warning in any run.

Synthetic data lets us measure what is predictable

σ = 5 MW

Demand noise, measured in eda.py. No model can beat an MAE of about 4.

~1%

of price steps get a random +100 spike, so spikes are unpredictable by design.

0.6 to 1.0

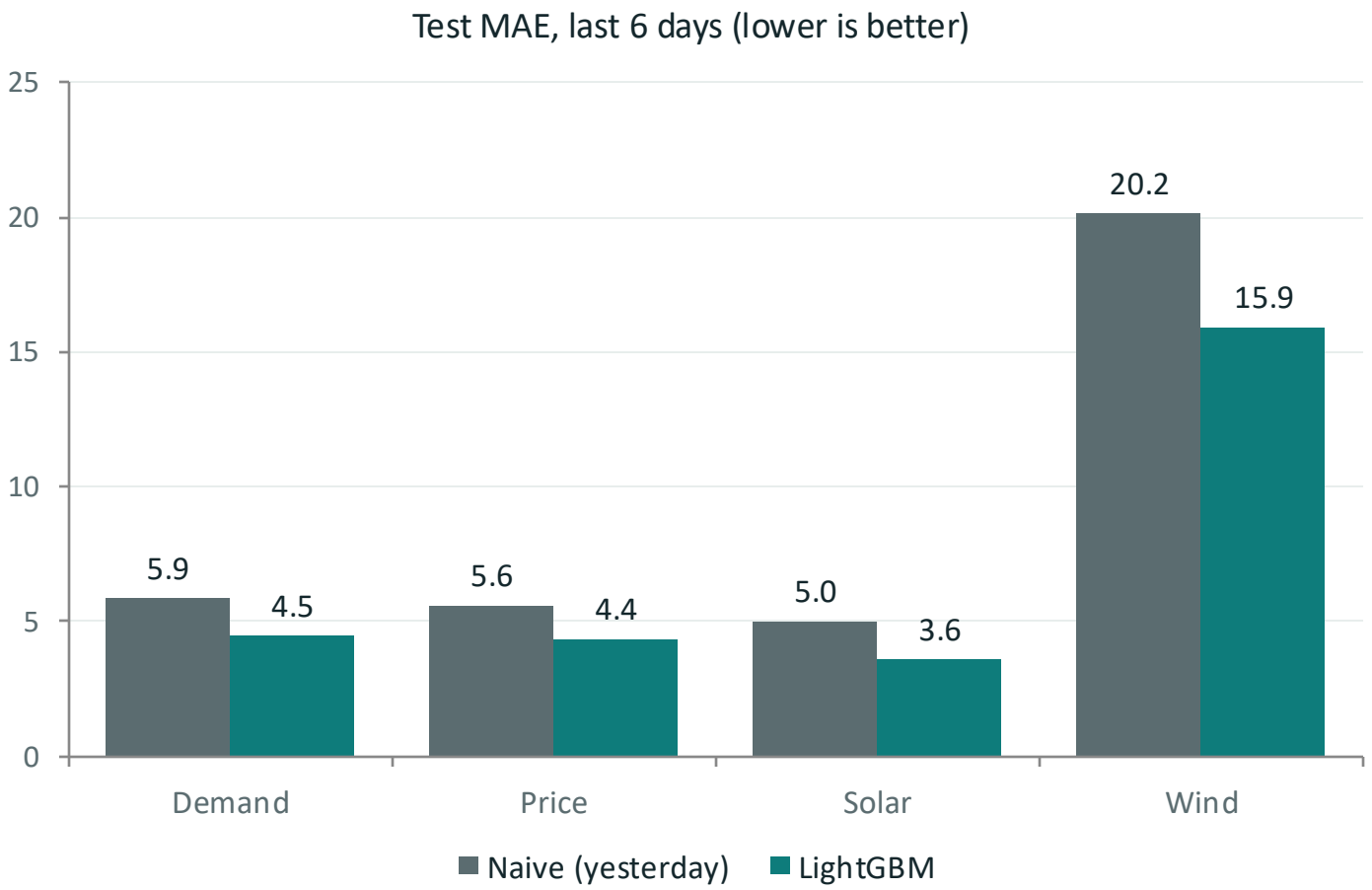
Solar cloud factor, redrawn every step. Only the daily shape is learnable.

30 days

2,880 steps of 15 minutes. Train on days 1 to 24, test on days 25 to 30.

Generated by make_data.py. Real weather and market data are future work.

LightGBM beats 'same as yesterday' on all four signals



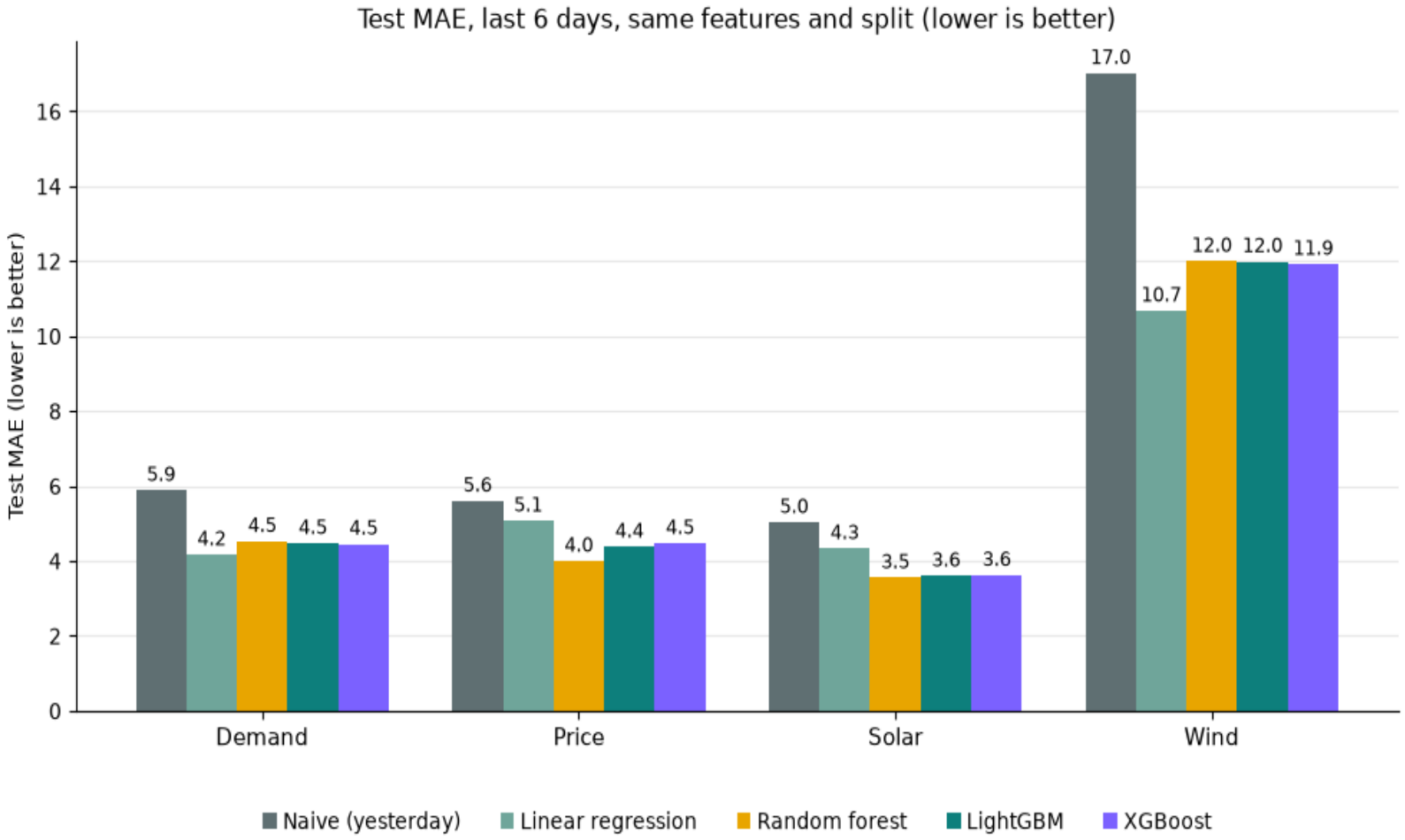
22 to 30%

lower MAE than the naive forecast. Lags are at least 24h old, so there is no leakage.

55.9 to 80%

p10 to p90 band coverage against an 80% target. Bands are overconfident, wind most of all.

Learned models all beat naive; the differences between them are small



Small differences

- **Linear Regression** best for demand (MAE 4.16) and wind (10.67)
- **Random Forest** best for price (3.99) and solar (3.55).
- **LightGBM** is close, but is not the best on every signal.

Why we kept LightGBM

LightGBM was competitive, but did not achieve the lowest MAE for every signal. We use it as a consistent forecasting pipeline and to generate quantile forecasts. Results are preliminary and based on synthetic data.

Baseline reacts to power balance; smart also reads price

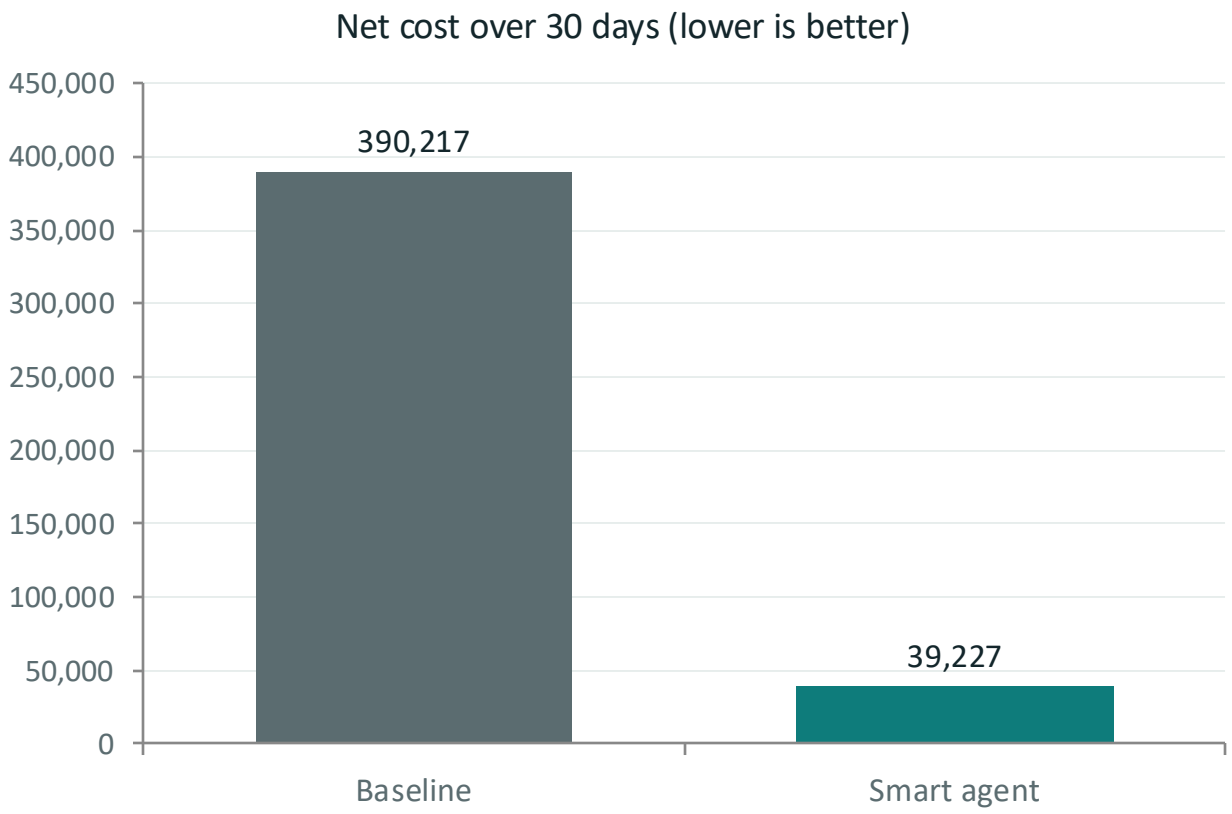
B Baseline (the number to beat)

- Surplus: charge the batteries
- Deficit: discharge the batteries
- Leftover surplus: sold at the current price
- Ignores price and battery level

S Smart agent (the one we present)

- Everything in the baseline, plus:
- Rule A: deficit at a cheap price, so buy from the grid and keep the battery charge
- Rule B: cheap price, no surplus, room in the batteries, so charge from the grid
- Optional: 'cheap' threshold from the 24h price forecast

Smart agent cuts net cost 90%, at the price of more carbon



-90%

net cost
390,217 to 39,227

+67%

carbon
7,780 to 13,016 t

Synthetic 30-day simulation; unlimited grid imports/exports and no market fees. Treat these results as illustrative, not a real-world savings forecast.

Carbon weight changes the best policy

Carbon weight	Best 'cheap' quantile	Net cost	Carbon (t)
0	0.60	35,580	14,186
0.005 to 0.05	**0.40**	**39,227**	**13,016**
0.10	0.10	331,528	8,181

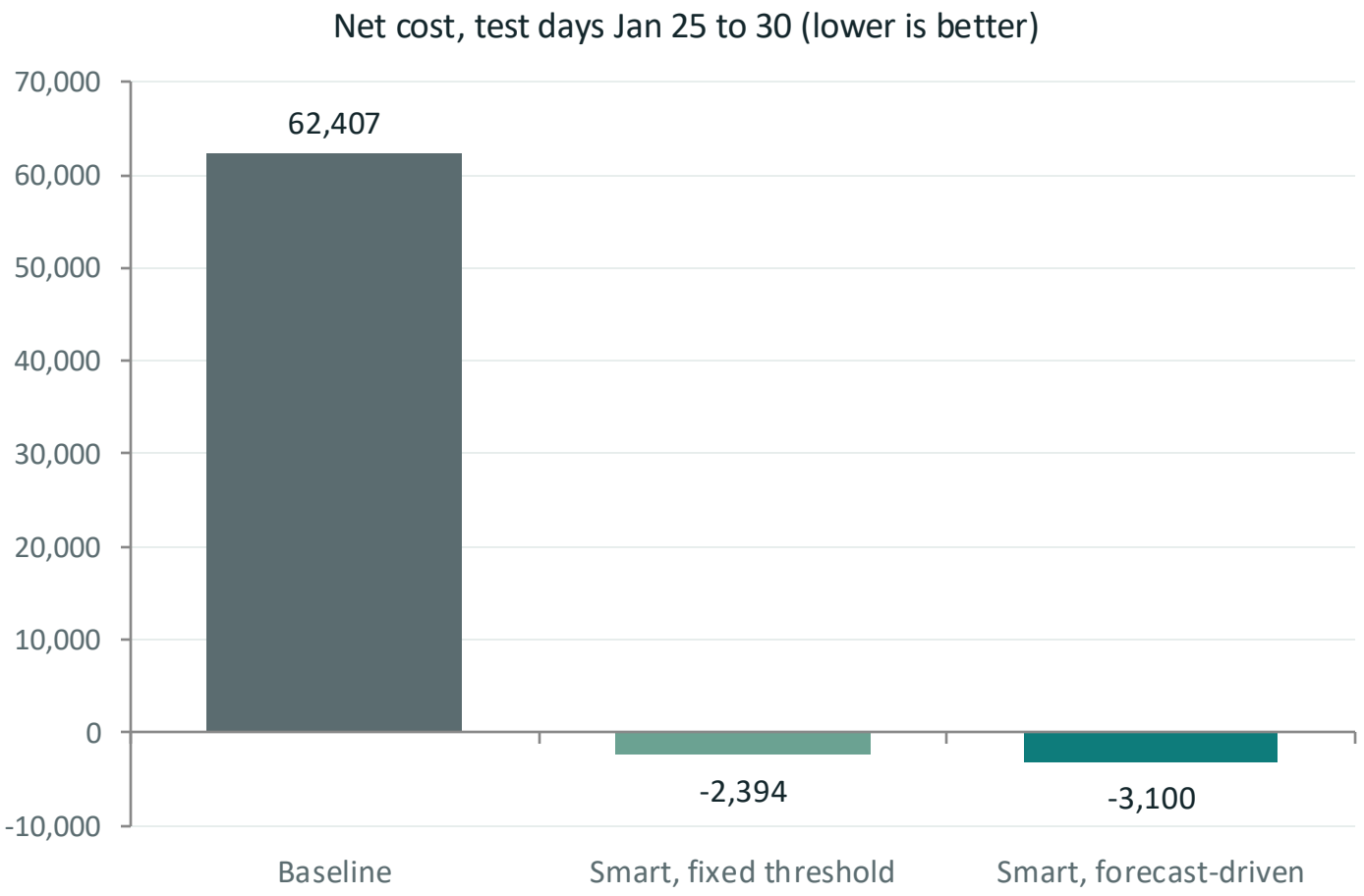
Score = net cost + carbon weight x carbon. A 4 x 6 grid search over the threshold and the weight picks the winner per weight.

Cost and carbon compete. At a very high weight the agent stops grid charging and cost jumps about 9x.

0.40

Recommended setting. It wins across a wide range of carbon weights, giving up about 3.6k of cost for about 1,170 t less carbon than the cost-only optimum.

Forecast-driven threshold beats a fixed one on unseen days



How to read it

- A replay of 6 days the forecaster never saw
- The fixed threshold uses all 30 days of prices, including these 6, so it peeks
- The forecast version uses predictions only
- Gap is about 700: modest on one run. Negative means sales exceeded purchases

Alerts change behavior: battery outage and storm reserve

Battery outage

+8%

B1 offline for one day: net cost 39,227 to 42,490. B2 and the grid cover the gap, and the balance check still holds.

Storm scenario	Min stored (MWh)	Net cost
No alert (same window)	**0.0**	39,227
Storm 18:00 to 22:00	**129.5**	39,014
Cancelled at 19:30	**185.8**	37,309
Extended to 01:00	**153.6**	45,077

Before a storm the agent tops up the batteries and stops draining them. During it, it keeps a 40% floor (120 MWh of 300) and buys the deficit from the grid instead.

Preparing is roughly cost-neutral here. A longer storm costs more (+5.9k for 7 hours), because the floor forces grid purchases.

The LLM interprets and explains, and never decides

1 Alert parser

'Battery B1 maintenance on Jan 12 for 2 days' becomes {start, end}. Gemini first, then Ollama, then a keyword fallback.

2 Storm parser

'Storm on Jan 12 from 18:00 to 22:00', 'stop at 19:30', 'extended until 01:00'. Pattern-matches supported alert formats. Ambiguous or unsupported alerts require operator review.Times in HH:MM.

3 Decision explainer

Receives the facts of one 15-minute step and writes two sentences. Falls back to a labelled template if no model answers.

Guardrail prompts carry what actually happened (charged, discharged, bought, sold), not only what the controller asked for. We caught labels that said 'charging' when the batteries were full, and fixed them at the source.

Challenges we hit and how we resolved them

Challenge	How we resolved it
First smart run looked too good, and the balance check warned	Grid power bought for charging was missing from purchases. Fixed, and the result became honest.
The explainer said 'charging' when batteries were full	The log now records what actually happened; the prompt includes charged, discharged, bought, sold.
Forecast was shown but never used; a fixed threshold peeks at future prices	Forecast-driven threshold, tested on 6 held-out days.
Storm reserve dipped to 110.8 MWh, below the 120 floor	Check the level after the planned discharge. Minimum is now 129.5 MWh.
Alert box read '9:00 PM' as 09:00; dashboard showed stale results	AM/PM parsing, stops before the storm ignored, storm passed into the cached run.
Forecast metrics differed between scripts (wind)	Reran and reconciled with one comparison script on the same split.
Final days worked alone against a fixed deadline	Cut MPC and demand response, and listed them as future work.

Limitations and future work

Limitations

• **Data:** synthetic. Net cost can be negative because the market has no fees and an unlimited grid.

• **Policy:** rule-based and tuned by search, not model predictive control. Forecasts feed one experiment, a forecast-driven price threshold on the last 6 days. The live dashboard uses a fixed threshold.

• **Curtailment:** a fallback for surplus that can neither be stored nor sold at an acceptable price. At the default setting it is zero.

• **Carbon:** counted on grid purchases only.

• **Alerts:** times are supplied before a run. There is no live mid-run replanning, and times must be typed as HH:MM.

•**Forecast bands:** overconfident (57 to 85% coverage against an 80% target).

•**Explanations:** the LLM describes decisions after the fact. It does not make them.

Future work

- MPC with probabilistic forecasts
- Live 15-minute step mode with mid-run alerts
- Grid export limit and line-limit events
- Calibrated bands; real weather and market data
- Demand response and load shifting, maintenance scheduling, battery degradation cost, line-limit and frequency events.

The current prototype demonstrates synthetic-data dispatch and scenario replay. It does not yet optimize dispatch under transmission constraints, guarantee supply during grid shortfalls, or perform live mid-run replanning,



Thank You!